

AKADEMIKA

EXPERIMENTAL MANUAL

WELCOME

Introduction

Today's system designers are faced with tomorrow's problems. EMBEDDED SYSTEM is one of the important subject need to teach while learning electronics. It is our vision to provide you with the product you need for training ensuring lasting reliability & quality.

Company's Motto

- Light years ahead – refers to leadership.

We strive to be the best in what we do and maintain high standards in the areas of

· Design · Quality · Value · Delivery · Support

We are indeed light years ahead of the competition in this field. We guarantee our valued customers great satisfaction.

Important Information

As you will move through this manual you will quickly discover that we have complete, truly innovative & superior training products. We are so committed to quality that we back our products with a complete and comprehensive warranty.

AKADEMIKA LAB SOLUTIONS

Sr. No. 15/8/1, Unit No.9, Kruti Industrial Estate, Opp.

Sangam Press, Kothrud, Pune, Maharashtra- 411038

INDIA

Contact No.:+91-9004904462

+91-7447438443

Email: lab@akademika.in

www.akademika.in

SAFETY RULES

- Read carefully and follow the instructions mentioned in this manual. This user manual includes all the important points about the installation, use and the maintenance of the product. Keep this manual always with you, for quick reference.
- After unpacking the product, arrange all the accessories in proper order, so that their integrity is checked with the packing list. Also, ensure that the accessories have no visible damage.
- Before connecting the power supply to the kit, be sure that the jumpers and the connecting chords are connected correctly, as per the experiment.
- This kit must be employed only for the use for which it has been conceived, i.e. as educational kit and must be used under the direct survey of expert personnel. Any other use is inadvisable and dangerous too. The manufacturer cannot be considered responsible for eventual damages due to improper, wrong or unreasonable uses.
- In case of any fault or malfunctioning in the trainer kit, turn off the power supply. Please do not tamper the kit. If you require our service, kindly contact the service center for technical assistance.
- The kits are liable to malfunction/under-perform if they are not operated under standard environmental conditions of temperature and humidity.

WARRANTY

This kit is warranted against defects in workmanship and materials. Any failure due to defect in either workmanship or material should occur under normal use within a year from the original date of purchase, such failure will be corrected free of charge to the purchaser by repair or replacement of defective part or parts. When the failure is result of user's neglect, natural disaster or accident, we would charge for repairs, regardless of the warranty period. The warranty does not cover include perishable items like connecting chords, crystals, etc. and other imported items.

Conditions and Limitations

The warranty is void and inapplicable if the defective product is not brought or sent to our authorized service center or sales outlet within the warranty period. Defective product will be AkademikaLab Solutions sole judgment. The defective product will be replaced with a new one or repaired, without charge or with charge.

In the warranty period if the service is needed, the purchaser should get in touch with the service center or the sales outlet. The purchaser should return the product to the service center or the sales outlet at his or her sole expense. The loss and damage in transit will be outside the preview of this warranty. A returned product must be accompanied by a written description of the defects. Type and Model No. of the kit has to be mentioned specifically. We return the product to the purchaser at our expense. In case the warranty does not cover the product on AkademikaLab Solutions judgment, we would repair the product after obtaining prior permission from purchaser who would receive an estimate statement about the repairing charges. In such cases, AkademikaLab Solutions bares the transporting expenses required to send back all the repaired products for the moment, and then repairs and transporting expenses will be charged against the purchaser by the statement of accounts.

When the authorized sales agents sell our products, they must notify the purchaser of the warranty contents, but they have no rights to stretch the meaning of original warranty contents or to offer an additional warranty. AkademikaLab Solutions does not provide any other promise or suggestive warranty and hold no liability for the damage caused by negligence, abnormal use or natural disaster. AkademikaLab Solutions is not responsible for the damages even if it is notified of above dangers in advance as well.

For more special service or overall repairs, maintenance and up gradation of products, be sure to contact our service center or the sales outlet.

INDEX

SR. NO	CHAPTER	PAGE NO
1.	Technical Specifications	5
2.	ARDUINO ROBOTIC ARM EXPERIMENTAL SETUP :Deliverables	7
3.	Introduction	12
4.	IDE INSTALLATION	13
5.	WALK THROUGH	21
6.	EXAMPLE 1 : TEST CODE	30
7.	EXAMPLE 2 : SIMPLE MOVEMENTS	33
8.	EXAMPLE 3 : PICKING UP A SPONGE	36

TECHNICAL SPECIFICATIONS :

ARDUINO UNO REV 3 SPECIFICATIONS :

Model Type	Arduino Uno R3
Microcontroller Chip	ATmega328P
Operating Voltage (VDC)	5
Input Voltage(Recommended)	7-12V
Analog I/O Pins	6
Digital I/O Pins	14 (of which 6 provide PWM output)
PWM Digital I/O Pins	6
Clock Speed	16 MHz
Flash Memory	32 KB
SRAM	2
EEPROM	1 KB (ATmega328P)
DC Current for 3.3V Pin (mA)	50
DC Current per I/O Pin (mA)	20
On Board LEDs	Yes
Operating Temperature (°C)	-10 to 60

Weight (gm)

25

TINKER ROBOTIC ARM KIT SPECIFICATIONS :

KIT INCLUDES :

Plastic Parts x 21
Screws x 63
Flat Washer x 16
Hexagon Nut x 7
Springs x 2
Servo Motors: 2 x SR 311, 4 x SR 431
Arduino compatible Shield x 1
Power Supply 5V, 4A x 1
Phillips Screwdriver x 1
Spiral Cable Protection Wrap x 1

Servo Technical Specification :

SpringRC SR431 - Dual Output Servo :

Control Signal
PWM Analog
Torque : @ 4.8V: 169.5 oz-in (12.2 kg-cm)
 : @ 6.0V: 201.4 oz-in (14.5 kg-cm)

Weight : 2.19 oz (62.0 g)

Dimensions : 1.65×0.81×1.56 in (42.0×20.5×39.5 mm)

Speed : @ 4.8V: 0.20 sec/60°
 : @ 6.0V: 0.18 sec/60°

Rotation Support : Dual Bearings

Gear Material : Metal

Rotation Range : 180°

Connector Type : J (aka Futaba)

SpringRC : SR311 :

Control Signal : PWM Analog

Torque : @ 4.8V: 43.13 oz-in (3.1 kg-cm)
: @ 6.0V: 52.86 oz-in (3.8 kg-cm)

Weight : 0.95 oz (27.0 g)

Dimensions : 1.23×0.65×1.13 in (31.3×16.5×28.6 mm)

Speed : @ 4.8V: 0.14 sec/60°
: @ 6.0V: 0.12 sec/60°

Rotation Support : Dual Bearings

Gear Material : Metal

Rotation Range : 180°

Connector Type : J (aka Futaba)

ARDUINO EXPERIMENTAL SETUP DELIVERABLES :

ARDUINO UNO REV 3 Qty 1

12V 1A DC Power supply Qty 1

USB A TO B cable Qty 1

TINKER ROBOTIC ARM KIT Qty 1

Pendrive with arduino IDE and source codes Qyt 1

DESCRIPTION :

1. **Power supply :-**
Input Voltage: AC 100 – 264V 50 / 60Hz.
Output Voltage: 12V DC, 1A, and 12Watt.
Output Voltage Type – DC Current.
Shell Material: Plastic (PVC)
Color: Black.
Weight: 160g Approx.
Power Connector Type: 3 Pin.
This Power Adaptor Has a 2.5mm x 5.5mm Output Pin Jack (Center Positive)
2. **USB Type: Type-A to Type-B :**
Weight: 25 gm.
Length: 50 cm.
Fully compatible with the PC.
Aluminum under-mold shield helps meet FCC requirements on KMI/RFI interference.
Foil and braid shield complies with fully rated cable specifications reducing EMI/FRI interference
3. **TINKER ROBOTIC ARM KIT :**
Plastic Parts x 21
Screws x 63
Flat Washer x 16
Hexagon Nut x 7
Springs x 2

Servo Motors: 2 x SR 311, 4 x SR 431
Arduino compatible Shield x 1
Power Supply 5V, 4A x 1
Phillips Screwdriver x 1
Spiral Cable Protection Wrap x 1

- 4. Arduino UNO REV 3 :**
Microcontroller Chip : ATmega328P
Operating Voltage (VDC) : 5
Input Voltage(Recommended) : 7-12V
Analog I/O Pins : 6
Digital I/O Pins : 14 (of which 6 provide PWM output)
PWM Digital I/O Pins : 6

INTRODUCTION

ARDUINO UNO REV 3 INTRODUCTION :

Arduino Uno is a microcontroller board based on the ATmega328P ([datasheet](#)). It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator (CSTCE16M0V53-R0), a USB connection, a power jack, an ICSP header and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started.. You can tinker with your Uno without worrying too much about doing something wrong, worst case scenario you can replace the chip for a few dollars and start over again.

"Uno" means one in Italian and was chosen to mark the release of Arduino Software (IDE) 1.0. The Uno board and version 1.0 of Arduino Software (IDE) were the reference versions of Arduino, now evolved to newer releases. The Uno board is the first in a series of USB Arduino boards, and the reference model for the Arduino platform; for an extensive list of current, past or outdated boards see the Arduino index of boards.

TINKER ROBOTIC ARM INTRODUCTION :

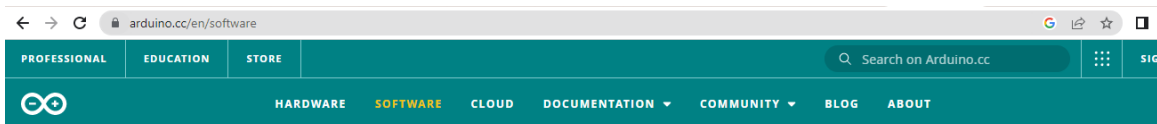
The TinkerKit Braccio is a fully operational robotic arm, controlled via Arduino. It can be assembled in several ways for multiple tasks such as moving objects. You can also attach a camera or solar panel. There are so many ways in which the Braccio can extend the reach of your devices.

SOFTWARE INSTALLATION :

Downloading and Installing the Arduino IDE 2.0 :

To download and install the Arduino IDE 2.0 on your Windows machine.

Download the editor from the Arduino Software page.



Downloads



Arduino IDE 2.0.3

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE
The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file
Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)
macOS Intel, 10.14: "Mojave" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits
[Release Notes](#)



Requirements

Windows - Win 10 and newer, 64 bits

The Arduino IDE 2.0

The Arduino IDE 2.0 is an open-source project. It is a big step from its sturdy predecessor, Arduino IDE 1.x, and comes with revamped UI, improved board & library manager, debugger, autocomplete feature and much more.

Download the Editor

Downloading the Arduino IDE 2.0 is done through the Arduino Software page. Here you will also find information on the other editors available to use.

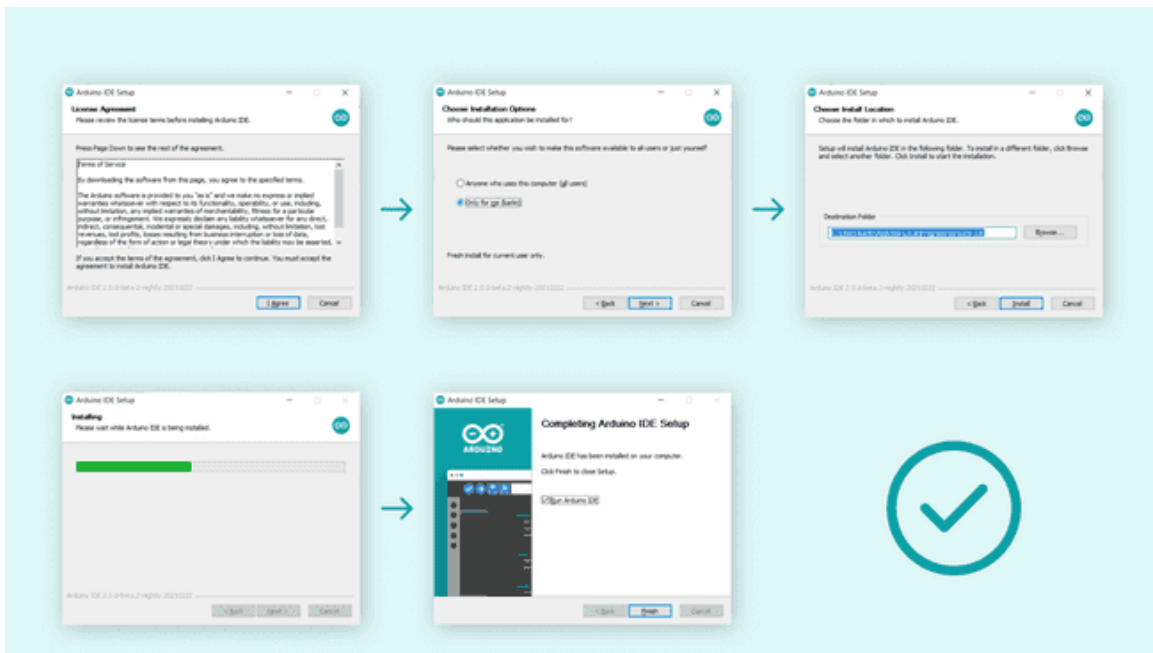
Installation

Windows

To install the Arduino IDE 2.0 on a Windows computer, simply run the file downloaded from the software page.

Running the installation file.

Follow the instructions in the installation guide. The installation may take several minutes.



Instructions for installing the IDE 2.0

Instructions for installing the IDE 2.0

You can now use the Arduino IDE 2.0 on your Windows computer!

HOW TO CREATE FIRST PROJECT IN ARDUINO IDE :

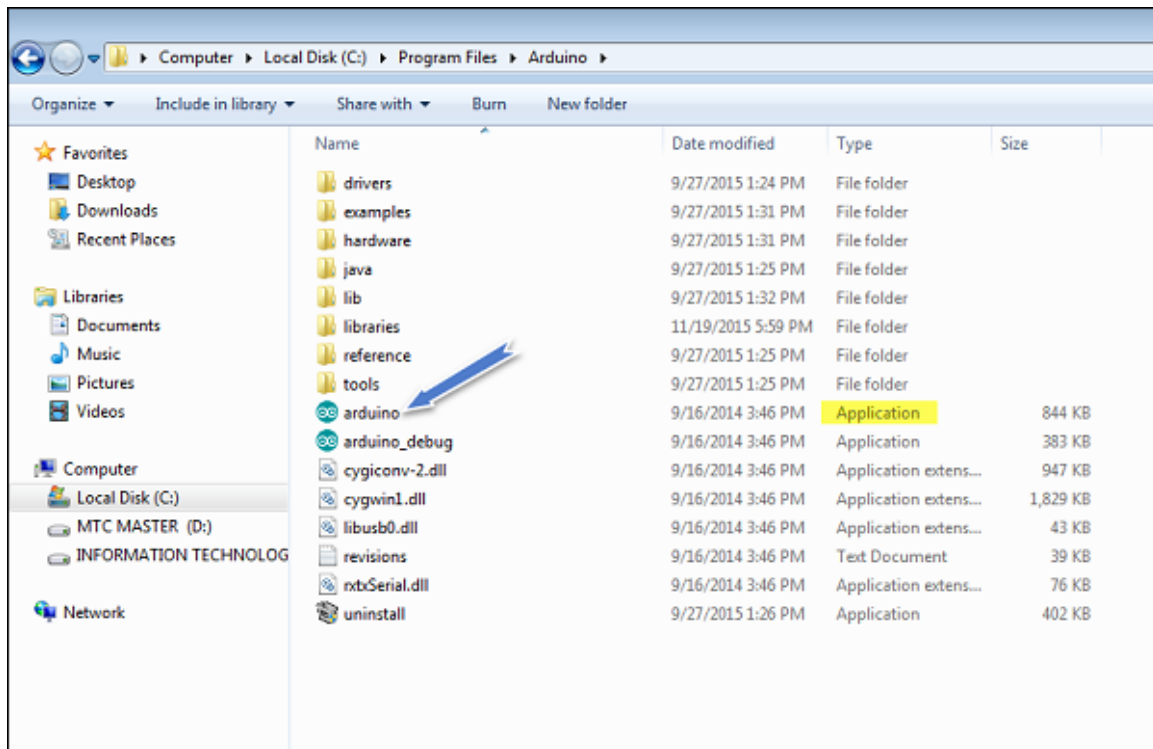
Step 1 : Power up your board.

The Arduino Uno, Mega, Duemilanove and Arduino Nano automatically draw power from either, the USB connection to the computer or an external power supply. If you are using an Arduino Diecimila, you have to make sure that the board is configured to draw power from the USB connection. The power source is selected with a jumper, a small piece of plastic that fits onto two of the three pins between the USB and power jacks. Check that it is on the two pins closest to the USB port.

Connect the Arduino board to your computer using the USB cable. The green power LED (labeled PWR) should glow.

Step 2 – Launch Arduino IDE.

After your Arduino IDE software is downloaded, you need to unzip the folder. Inside the folder, you can find the application icon with an infinity label (application.exe). Double-click the icon to start the IDE.



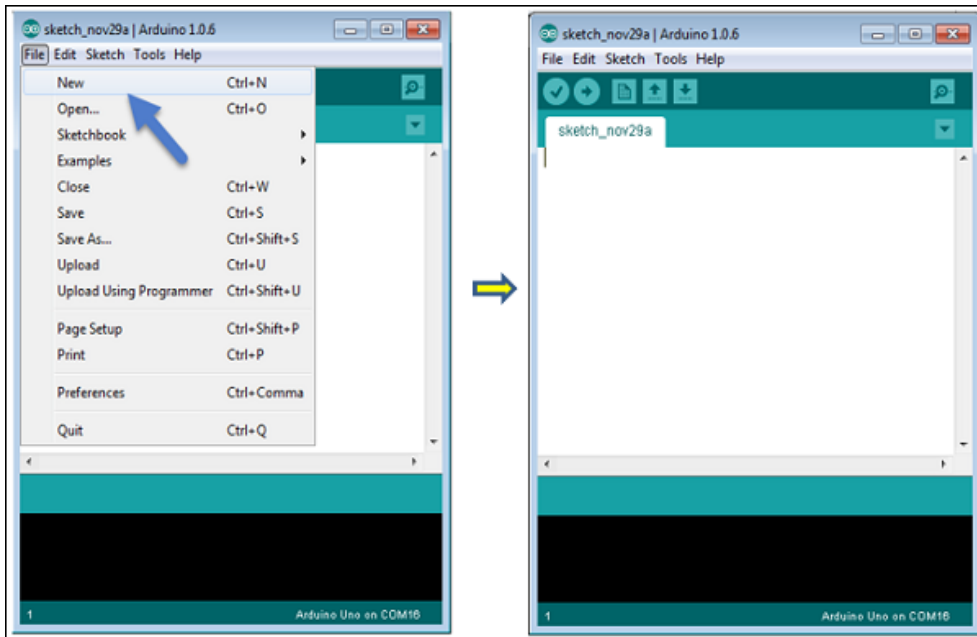
Step 3 – Open your first project.

Once the software starts, you have two options –

Create a new project.

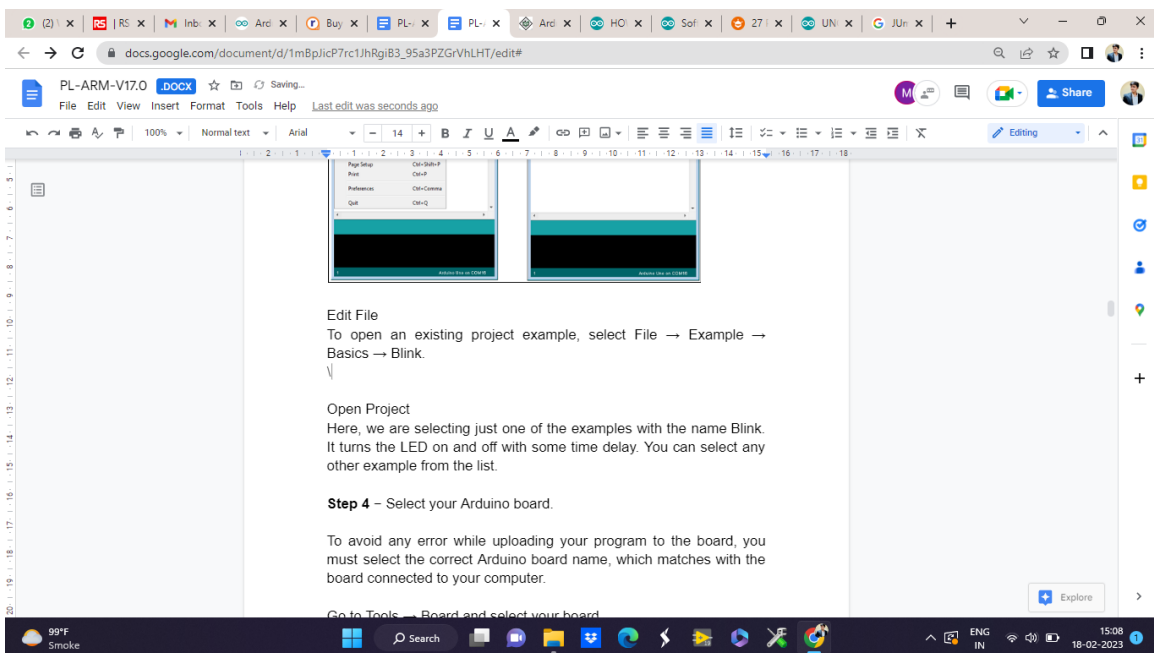
Open an existing project example.

To create a new project, select File → New.



Edit File

To open an existing project example, select File → Example → Basics → Blink.

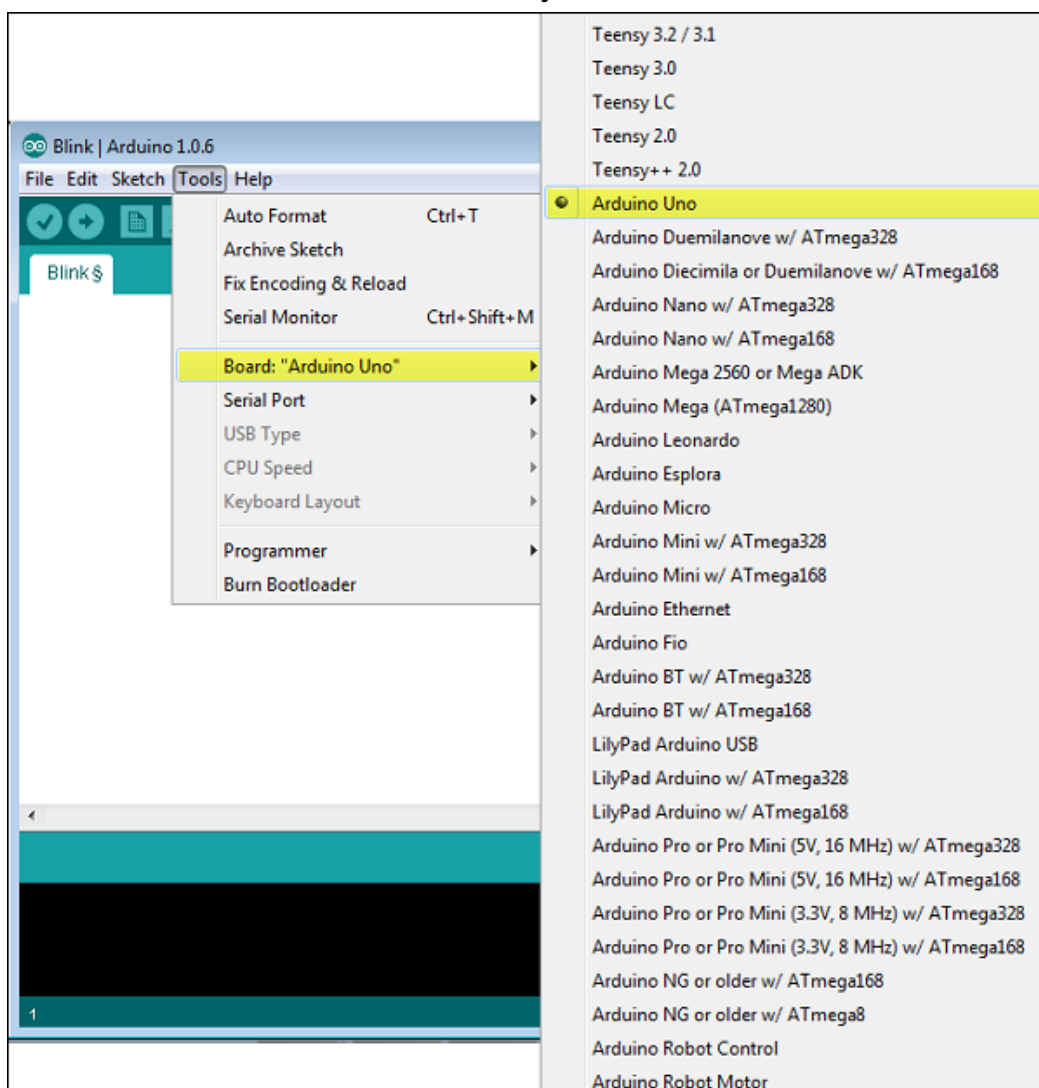


Here, we are selecting just one of the examples with the name Blink. It turns the LED on and off with some time delay. You can select any other example from the list.

Step 4 – Select your Arduino board.

To avoid any error while uploading your program to the board, you must select the correct Arduino board name, which matches with the board connected to your computer.

Go to Tools → Board and select your board.

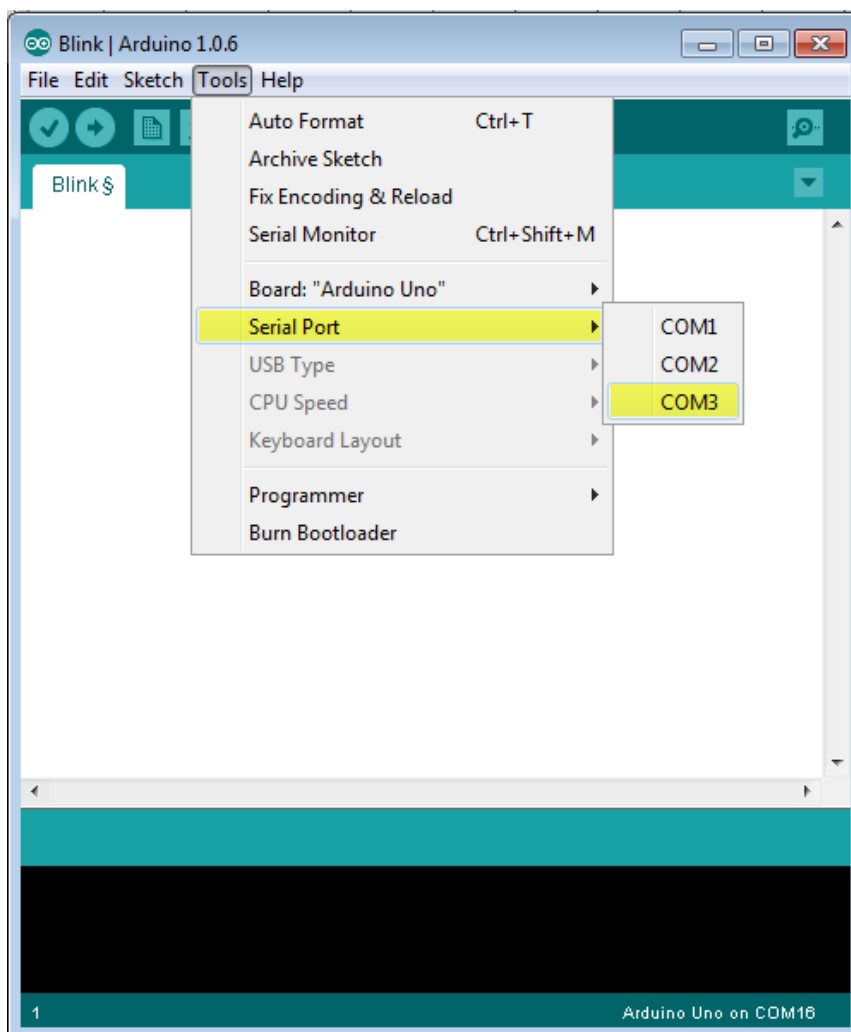


Select Board

Here, we have selected Arduino Uno board according to our tutorial, but you must select the name matching the board that you are using.

Step 5 – Select your serial port.

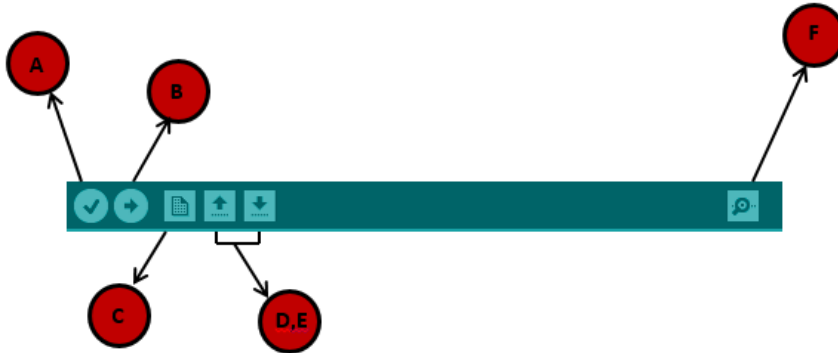
Select the serial device of the Arduino board. Go to Tools → Serial Port menu. This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports). To find out, you can disconnect your Arduino board and re-open the menu, the entry that disappears should be of the Arduino board. Reconnect the board and select that serial port.



Serial Port

Step 6 – Upload the program to your board.

Before explaining how we can upload our program to the board, we must demonstrate the function of each symbol appearing in the Arduino IDE toolbar.

**Upload program**

A – Used to check if there is any compilation error.

B – Used to upload a program to the Arduino board.

C – Shortcut used to create a new sketch.

D – Used to directly open one of the example sketch.

E – Used to save your sketch.

F – Serial monitor used to receive serial data from the board and send the serial data to the board.

Now, simply click the "Upload" button in the environment. Wait a few seconds; you will see the RX and TX LEDs on the board, flashing. If the upload is successful, the message "Done uploading" will appear in the status bar.

INTRODUCTION :

The TinkerKit Braccio is a fully operational robotic arm, controlled via Arduino. It can be assembled in several ways for multiple tasks such as moving objects. You can also attach a camera or solar panel. There are so many ways in which the Braccio can extend the reach of your devices.

ASSEMBLY :

Before you can use Tinkerkit Braccio you need to assemble it, so take your time, find a proper desk with enough space to place all the parts comfortably and follow our instructions.

You may follow our printed guide available inside the box, or you can download an electronic copy.

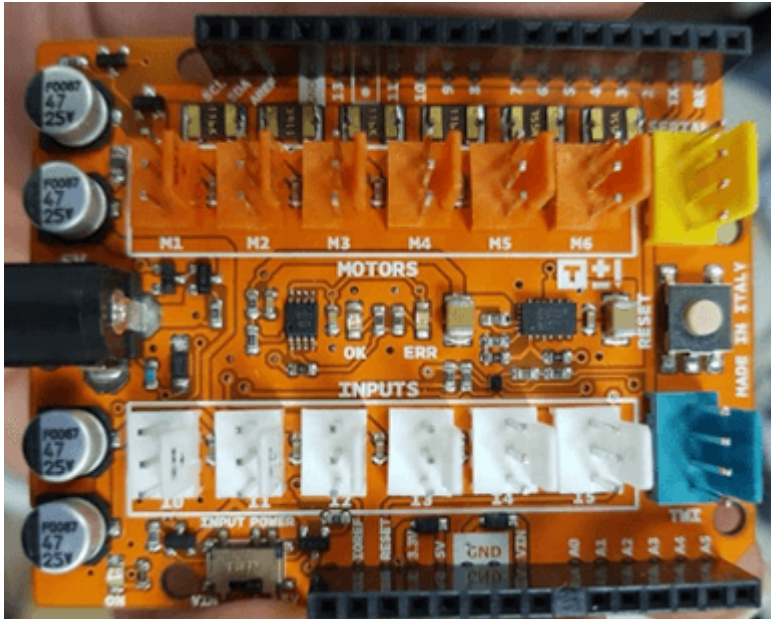
If you prefer to see all the steps to build it we have made a video that you can play, stop and repeat as you need.

BRACCIO SHIELD :

The included Braccio shield allows you to hook up the servos directly to your Arduino board.

Braccio Shield V4 :

The latest version of Braccio shield is V4; you can recognize it by the power switch that is necessary to properly connect Arduino Yun, Tian and Primo: "VIN" position for Yun, Tian and Primo, "5V" for the other boards.



For a correct use of the Braccio shield V4 is necessary at least the IDE 1.7.11.

On the Braccio shield V4 a new feature is present: the "Soft-start". This function is included in the

Braccio.begin()

Function and starts the Braccio softly and safe to protect the servo motors for the first 8 seconds.

The Soft-start works each time you call

Braccio.begin(). During this phase the shield doesn't give the maximum voltage level to the servo motors and you should not use the Braccio to lift any weight.

If the Braccio hit something or is locked the resettable fuse on the Braccio shield could detach the power for the servo motor and is necessary to unplug and replug the power connector of the Braccio shield.

With the Braccio Shield V4 the digital 12 is not available if you are connecting the Braccio to the shield or if you are using M1, M2, M3, M4, M5, **M6 connectors as digital.**

Pin 12 is used to manage the voltage level of the Soft-start (on the shield there's a dot instead of "12")

If you want to disable the Soft-start function:

```
pinMode(12, OUTPUT); // you need to set HIGH the pin 12
digitalWrite(12, HIGH);
Braccio.begin(SOFT_START_DISABLED); and set a proper parameter to disable the soft start
```

In this way you can normally use M1-M6 connectors as digital and you can use Braccio without the soft start.

The shield has 12 standard TinkerKit 3-pin connectors.

The connectors labeled M1 through M6 are connected to the PWM capable outputs of the Arduino board. The connectors labeled I0 through I5 are analog inputs.

Each of the motor connections have a protection fuse:

- M1 through M4 are limited to 1.1A
- M5 and M6 are limited to 750mA

The connectors are connected to the following pins on the shield:

Connector Name	Shield pin
M1	11
M2	10
M3	9
M4	6
M5	5
M6	3
I0	A0 (aka 14)
I1	A1 (15)
I2	A2 (16)
I3	A3 (17)
I4	A4 (18)
I5	A5 (19)
TWI	SCL, SDA

Serial	RX0, TX0
--------	-------------

The 4-pin TWI connector allows your Arduino board to communicate with devices that support the TWI (Two Wire Interface) or I2C (Inter-Integrated Circuit) protocol through the Wire library in Arduino. Ground and 5 volt connections are provided on the connector.

NOTE: On the Arduino UNO, the I2C bus uses Analog Input 4 and 5 (A4 and A5), whereas on the Arduino Mega the I2C connection is on pins 20 and 21. Using the TWI connector on the UNO (and similar) will preclude the use of those analog inputs (A4 and A5).

The 4-pin SERIAL connector allows the board to communicate with other devices that support serial communication. Ground and 5 volt connections are provided on the connector for your convenience.

NOTE: If you're sending or receiving data to and from the computer this serial connector is not available.

The shield contains three status LEDs:

- Green LED (LED ON) signals that the shield is on
- Another green LED (LED OK) signals that the shield is correctly powered
- A red LED (LED ERR) signals that the shield is not correctly powered

A standard 6 mm pushbutton allows you to reset the board, and there is a jack for the external 5V 5000 mA power supply.

Compatible Boards:

- Uno
- Uno SMD
- Uno WiFi
- Due
- Mega 2560
- Ethernet
- Leonardo
- Leonardo ETH
- M0
- M0 Pro
- Yun*
- Tian*

Using the Arduino UNO WiFi, Leonardo Ethernet, Yún or Tian, you can control your Braccio through the network with your PC or your smartphone. See below for a demonstration.

Note for Tian or Yun.

Take care plugging the shield on Tian or Yun, do not push the shield deeply due to not create a short circuit between the M1 connector and the ethernet plug.

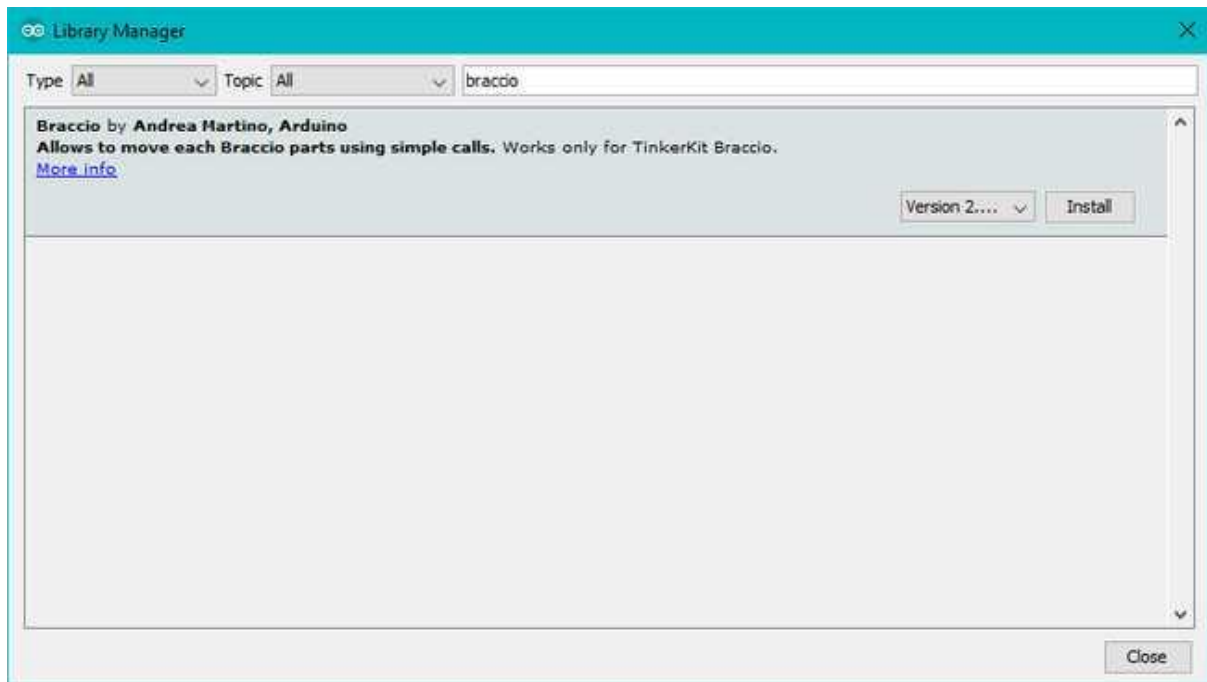
Note for the Arduino YÚN, Tian and Primo with Braccio Shield

V4

To make its compatible with the shield, you must move the power switch to "VIN" position due to the fact that the Arduino Yun, Tian and Primo cannot be powered from the 5V pin.

TinkerKit Braccio Library

To use your TinkerKit Braccio you need to load the Braccio library using the Library manager of the Arduino Software (IDE); see [these instructions](#) for details on installing libraries.



The functions available in this library are just two.

Braccio.Begin()

Initialize and set up the initial position for Braccio

All the servo motors will be positioned in the "safety" position:

Base (M1): 90 degrees

Shoulder (M2): 45 degree

Elbow (M3): 180 degrees

Wrist vertical (M4): 180 degrees

Wrist rotation (M5): 90 degrees

Gripper (M6): 10 degrees

Braccio.ServoMovement (step delay, M1, M2, M3, M4, M5, M6)

With this function you can control all the Braccio's servos with only one command

Step Delay: a milliseconds delay between the movement of each servo. Allowed values from 10 to 30 msec.

M1: base degrees. Allowed values from 0 to 180 degrees

M2: shoulder degrees. Allowed values from 15 to 165 degrees

M3: elbow degrees. Allowed values from 0 to 180 degrees

M4: wrist vertical degrees. Allowed values from 0 to 180 degrees

M5: wrist rotation degrees. Allowed values from 0 to 180 degrees

M6: gripper degrees. Allowed values from 10 to 73 degrees. 10: the tongue is open, 73: the gripper is closed

EXAMPLE NO : 1 TEST BRACCIO

TEST BRACCIO :

/*

testBraccio90.ino

testBraccio90 is a setup sketch to check the alignment of all the servo motors

This is the first sketch you need to run on Braccio

When you start this sketch Braccio will be positioned perpendicular to the base

If you can't see the Braccio in this exact position you need to realign the servo motors position

Created on 18 Nov 2015

by Andrea Martino

This example is in the public domain.

*/

#include <Braccio.h>


```
#include <Servo.h>

Servo base;

Servo shoulder;

Servo elbow;

Servo wrist_rot;

Servo wrist_ver;

Servo gripper;

void setup() {

    //Initialization functions and set up the initial position for Braccio

    //All the servo motors will be positioned in the "safety" position:

    //Base (M1):90 degrees

    //Shoulder (M2): 45 degrees

    //Elbow (M3): 180 degrees

    //Wrist vertical (M4): 180 degrees

    //Wrist rotation (M5): 90 degrees

    //gripper (M6): 10 degrees

    Braccio.begin();
```

```
}
```

```
void loop() {
```

```
  /*
```

Step Delay: a milliseconds delay between the movement of each servo.

Allowed values from 10 to 30 msec.

M1=base degrees. Allowed values from 0 to 180 degrees

M2=shoulder degrees. Allowed values from 15 to 165 degrees

M3=elbow degrees. Allowed values from 0 to 180 degrees

M4=wrists vertical degrees. Allowed values from 0 to 180 degrees

M5=wrists rotation degrees. Allowed values from 0 to 180 degrees

M6=gripper degrees. Allowed values from 10 to 73 degrees. 10: the tongue is open, 73: the gripper is closed.

```
  */
```

```
  // the arm is aligned upwards and the gripper is closed
```

```
    //(step delay, M1, M2, M3, M4, M5, M6);
```

```
    Braccio.ServoMovement(20, 90, 90, 90, 90, 90, 73);
```

```
}
```

EXAMPLE NO : 2 SIMPLE MOVEMENTS

SIMPLE MOVEMENT :

```
/*
```

```
simpleMovements.ino
```

This sketch simpleMovements shows how they move each servo motor of Braccio

Created on 18 Nov 2015

by Andrea Martino

This example is in the public domain.

```
*/
```

```
#include <Braccio.h>
```

```
#include <Servo.h>
```

```
Servo base;
```

```
Servo shoulder;
```

```
Servo elbow;
```

```
Servo wrist_rot;
```

```
Servo wrist_ver;
```

```
Servo gripper;
```

```
void setup() {
```

```
    //Initialization functions and set up the initial position for Braccio
```

```
    //All the servo motors will be positioned in the "safety" position:
```

```
    //Base (M1):90 degrees
```

```
    //Shoulder (M2): 45 degrees
```

```
    //Elbow (M3): 180 degrees
```

```
    //Wrist vertical (M4): 180 degrees
```

```
    //Wrist rotation (M5): 90 degrees
```

```
    //gripper (M6): 10 degrees
```

```
    Braccio.begin();
```

```
}
```

```
void loop() {
```

```
    /*
```

```
        Step Delay: a milliseconds delay between the movement of each servo.
```

```
        Allowed values from 10 to 30 msec.
```

M1=base degrees. Allowed values from 0 to 180 degrees

M2=shoulder degrees. Allowed values from 15 to 165 degrees

M3=elbow degrees. Allowed values from 0 to 180 degrees

M4=wrst vertical degrees. Allowed values from 0 to 180 degrees

M5=wrst rotation degrees. Allowed values from 0 to 180 degrees

M6=gripper degrees. Allowed values from 10 to 73 degrees. 10: the
tounge is open, 73: the gripper is closed.

*/

//(step delay, M1, M2, M3, M4, M5, M6);

Braccio.ServoMovement(20, 0, 15, 180, 170, 0, 73);

//Wait 1 second

delay(1000);

Braccio.ServoMovement(20, 180, 165, 0, 0, 180, 10);

//Wait 1 second

delay(1000);

}

EXAMPLE NO : 3 TAKE THE SPONGE

TAKE THE SPONGE :

```
/*
```

```
  takethesponge.ino
```

This example commands to the Braccio to take a sponge from the table and it shows to the user

Created on 18 Nov 2015

by Andrea Martino

This example is in the public domain.

```
*/
```

```
#include <Braccio.h>
```

```
#include <Servo.h>
```

```
Servo base;
```

```
Servo shoulder;
```

```
Servo elbow;
```

```
Servo wrist_rot;
```

```
Servo wrist_ver;
```

```
Servo gripper;
```

```
void setup() {
```

```
    //Initialization functions and set up the initial position for Braccio
```

```
    //All the servo motors will be positioned in the "safety" position:
```

```
    //Base (M1):90 degrees
```

```
    //Shoulder (M2): 45 degrees
```

```
    //Elbow (M3): 180 degrees
```

```
    //Wrist vertical (M4): 180 degrees
```

```
    //Wrist rotation (M5): 90 degrees
```

```
    //gripper (M6): 10 degrees
```

```
    Braccio.begin();
```

```
}
```

```
void loop() {
```

```
    /*
```

```
        Step Delay: a milliseconds delay between the movement of each servo.
```

```
        Allowed values from 10 to 30 msec.
```

```
        M1=base degrees. Allowed values from 0 to 180 degrees
```

M2=shoulder degrees. Allowed values from 15 to 165 degrees

M3=elbow degrees. Allowed values from 0 to 180 degrees

M4=wrists vertical degrees. Allowed values from 0 to 180 degrees

M5=wrists rotation degrees. Allowed values from 0 to 180 degrees

M6=gripper degrees. Allowed values from 10 to 73 degrees. 10: the tongue is open, 73: the gripper is closed.

*/

//Starting position

//(step delay M1 , M2 , M3 , M4 , M5 , M6);

Braccio.ServoMovement(20, 0, 45, 180, 180, 90, 10);

//Wait 1 second

delay(1000);

//The braccio moves to the sponge. Only the M2 servo will move

Braccio.ServoMovement(20, 0, 90, 180, 180, 90, 10);

//Close the gripper to take the sponge. Only the M6 servo will move

Braccio.ServoMovement(10, 0, 90, 180, 180, 90, 60);

//Brings the sponge upwards.

```
Braccio.ServoMovement(20, 0, 45, 180, 45, 0, 60);

//Show the sponge. Only the M1 servo will moves

Braccio.ServoMovement(20, 180, 45, 180, 45, 0, 60);

//Return to the start position.

Braccio.ServoMovement(20, 0, 90, 180, 180, 90, 60);

//Open the gripper

Braccio.ServoMovement(20, 0, 90, 180, 180, 90, 10 );

}
```

